

EXAM LAST MIN REVISION UNIT 6

Hibernate vs JDBC

JDBC	Hibernate
Manual SQL	Automatic SQL
More Code	Less Code
Manual Mapping	Automatic Mapping
More Development Time	Faster Development
Database Dependent	More Database Independent

Exam Points

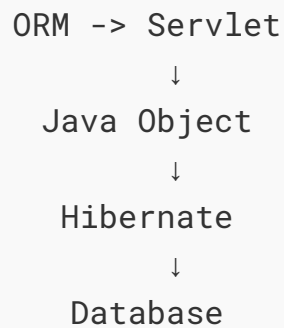
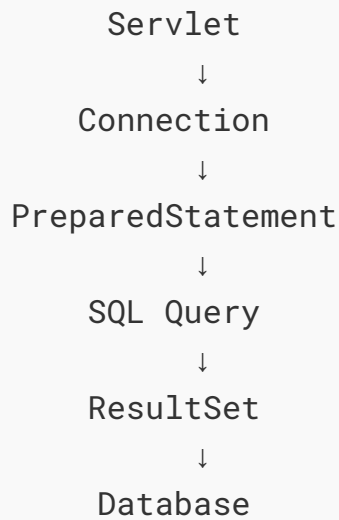
- Hibernate is an ORM framework.
- Used to simplify database operations.
- Maps Java objects to database tables.
- Reduces JDBC code.
- Generates SQL automatically.
- Supports CRUD operations.
- Provides database independence.

QUICK REVISION (2 MARKS)

Hibernate is an ORM (Object Relational Mapping) framework for Java that simplifies database operations by mapping Java objects to database tables. It reduces JDBC code, automatically generates SQL queries, and provides efficient interaction with relational databases.

JDBC VS ORM

JDBC flow vs ORM flow



JDBC	ORM (Hibernate)
Manual SQL	Automatic SQL
More Coding	Less Coding
Manual Mapping	Automatic Mapping
Database Dependent	More Database Independent
No Built-in Cache	Cache Support

JDBC	ORM (Hibernate)
More Development Time	Faster Development
Lower Level API	Higher Level Framework

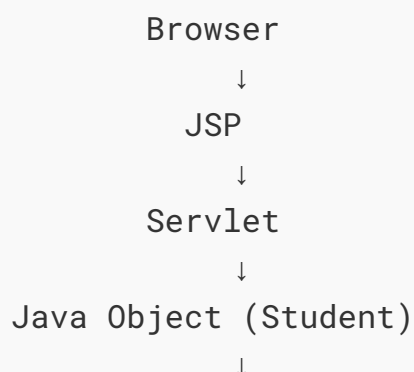
Most Important Exam Points

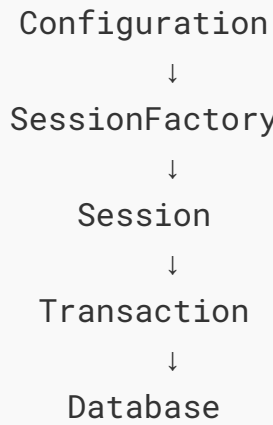
- ORM maps Java objects to database tables.
- Hibernate is an ORM framework.
- Reduces JDBC code.
- Automatic object-table mapping.
- Database independence.
- Easier maintenance.
- Supports caching and transaction management.
- Improves productivity.

QUICK REVISION (2 MARKS)

ORM provides automatic mapping between Java objects and database tables, reducing the need for manual SQL and JDBC code. Compared to JDBC, ORM frameworks such as Hibernate offer less coding, automatic mapping, easier maintenance, database independence, transaction management, and caching support.

Hibernate Architecture





Why So Many Components?

Suppose you want to save:

ID = 2

Name = Indra

Hibernate needs to know:

- Which Database?
- Username?
- Password?
- Which Table?
- How to Connect?

Configuration handles this.

Component	Exact Purpose
Configuration	Reads Hibernate settings
SessionFactory	Creates Session objects
Session	Performs CRUD operations
Transaction	Commit/Rollback and data consistency
Query	Retrieves/manipulates data using HQL

Exam Points

- Hibernate architecture consists of Configuration, SessionFactory, Session, Transaction, and Query.
- Configuration reads settings.
- SessionFactory creates Session objects.
- Session performs CRUD operations.
- Transaction manages commit and rollback.
- Query retrieves data from database.

QUICK REVISION (2 MARKS)

Hibernate Architecture consists of Configuration, SessionFactory, Session, Transaction, and Query. Configuration loads settings, SessionFactory creates Sessions, Session performs database operations, Transaction ensures data consistency, and Query retrieves data from the database.

Setting Up Hibernate in a Java Application

Setting up Hibernate in a Java application involves adding Hibernate libraries and the JDBC driver to the project, creating an entity class, configuring database settings in the hibernate.cfg.xml file, and mapping Java classes to database tables using XML or annotations. After configuration, a SessionFactory and Session are created to interact with the database. Proper setup enables Hibernate to manage database operations such as insert, update, delete, and retrieval without requiring extensive JDBC code. This process establishes communication between the Java application and the relational database.

Mapping Java Classes to Database Tables

XML vs Annotations

XML Configuration	Annotation Configuration
Separate file	Written inside class
More code	Less code

XML Configuration	Annotation Configuration
Harder to maintain	Easier to maintain
Older approach	Modern approach
Mapping outside class	Mapping inside class

1. Create hibernate.cfg.xml
- ↓
2. Add DB URL, username, password
- ↓
3. Register Entity Classes
- ↓
4. Create SessionFactory
- ↓
5. Open Session
- ↓
6. Perform CRUD Operations

Exam Definition (5 Marks)

Hibernate Configuration is the process of providing database connection details, Hibernate properties, and mapping information to Hibernate. It is usually done using the hibernate.cfg.xml file or annotations. Hibernate reads this configuration to create a SessionFactory and establish communication with the database.

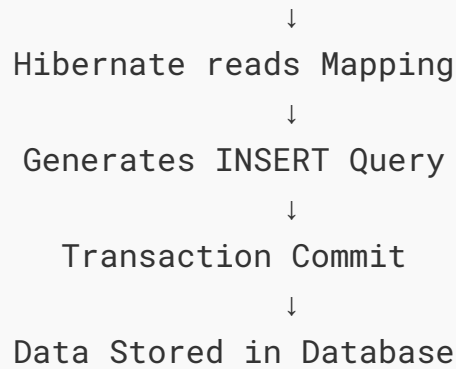
BASIC CRUD OPERATION

Save Operation Flow

```

Create Object
    ↓
session.save(object)

```



Exam-Oriented Definition

The Save operation in Hibernate is used to insert a new object into the database. The `save()` method converts a Java object into a database record and stores it permanently after the transaction is committed.

UPDATE

Exam-Oriented Definition

The Update operation in Hibernate is used to modify an existing record in the database. The `update()` method updates the row corresponding to the object's primary key and permanently saves the changes after the transaction is committed.

RETRIEVE

Exam Definition

Retrieve (Read) operation in Hibernate is used to fetch data from the database and convert it into Java objects. The `get()` method retrieves a record using its primary key and returns the corresponding object.

Hibernate Query Language (HQL)

Hibernate Query Language (HQL) is an object-oriented query language provided by Hibernate to interact with the database. Unlike traditional SQL, which works with database tables and

columns, HQL works with Java entity classes and their properties. Hibernate automatically converts HQL queries into database-specific SQL queries, making applications database-independent and easier to maintain. HQL is mainly used to retrieve, update, and delete data using Java objects rather than directly accessing tables.

Common HQL Syntax and Equivalent SQL

Purpose	HQL	SQL
Fetch all records	FROM Student	SELECT * FROM Student;
Fetch records with condition	FROM Student WHERE age = 21	SELECT * FROM Student WHERE age = 21;
Retrieve specific field	SELECT name FROM Student	SELECT name FROM Student;
Update record	UPDATE Student SET age = 22 WHERE id = 101	UPDATE Student SET age = 22 WHERE id = 101;
Delete record	DELETE FROM Student WHERE id = 101	DELETE FROM Student WHERE id = 101;

Key Difference: In SQL, Student, name, age, and id refer to table names and column names, whereas in HQL, Student, name, age, and id refer to Java entity class names and their properties. Hibernate automatically translates HQL into SQL before executing it on the database. For example, HQL FROM Student works on the Java class Student, while SQL SELECT * FROM Student works directly on the database table Student.

HIBERNATE CRUD

Hibernate CRUD Quick Revision

INSERT:

Object → session.save(obj) → tx.commit()

READ ONE:

session.get(ClassName.class, id)

READ ALL:

session.createQuery("from ClassName") → list()

UPDATE:

session.get(ClassName.class, id) → modify object → session.update(obj) → tx.commit()

DELETE:

`session.get(ClassName.class, id) → session.delete(obj) → tx.commit()`

Note:

- `commit()` is required for `save()`, `update()`, and `delete()` because they modify the database.
- `commit()` is usually not required for `get()` and `createQuery()` because they only read data.